SR-MPLS Training Lab (vJunos / GNS3)

Self-paced. Work each module in order; every module builds on the one before it. Do not skip the Brief it back checkpoints, they are how you confirm mastery before the next layer. Command examples use PE1 unless noted; repeat each step on every node using its own values from configs/stage<N>/<node>.set, or push a whole stage at once with 'python push_lab_configs.py stage<N>-<name>'.

M0 Junos Fundamentals

Estimated time: 45 min

Objective

Get comfortable driving Junos before touching Segment Routing. You will move between operational and configuration mode, use the candidate / commit model safely, recover with rollback, and read how the lab delivers its mandatory settings through configuration groups. By the end you can confirm PE1 is sitting at a clean, committed base state.

Before you start: The GNS3 project is imported and PE1 has booted with the stage0-base config loaded (host-name, fxp0 management address, loopback, and transit interfaces). If you are building from bare metal, console in and load configs/stage0-base/PE1.set first.

Concept primer

Junos keeps two separate configuration databases. The active (committed) database is what the router is actually running. The candidate database is a private scratch copy where every 'set' and 'delete' lands. Nothing you type changes the running network until you issue 'commit', which validates the candidate and atomically swaps it in. This is very different from IOS, where 'conf t' edits the running config live and 'write memory' only saves it.

Two safety nets follow from that model. 'commit confirmed <minutes>' commits the candidate but starts a timer; if you do not type a second plain 'commit' before the timer expires, the router automatically rolls back. This is how you make risky remote changes without locking yourself out. 'rollback <n>' loads a previous committed snapshot (0 is the current committed config) back into the candidate so you can discard mistakes.

Configuration groups let you define a block of settings once and apply it to many places with 'apply-groups'. This lab uses them for its immutable mandates. The MTU group sets 'interfaces <ge-*> mtu 9192', where <ge-*> is a wildcard matching every ge- interface, so one group line enforces jumbo MTU fabric-wide. Later modules add the ISIS-PARAMETERS group the same way. You must never negate these groups; they are the ground truth of the design.

Build steps

1. **Enter the CLI and move between modes.**

```
lab@PE1> configure
Entering configuration mode
[edit]
lab@PE1# run show system uptime    ## 'run' calls an operational command from config mode
lab@PE1# top                       ## jump to the config root
```

Note: The '>' prompt is operational mode (show/ping/traceroute). The '#' prompt is configuration mode. 'run' lets you execute an operational command without leaving the candidate.

2. **Read the committed base config as flat set-lines.**

```
lab@PE1# run show configuration | display set | match host-name
```

Note: '| display set' turns the hierarchical config into the same set-command form used in the stage files, which makes it easy to compare against configs/stage0-base/PE1.set.

3. **Practice a safe change with an auto-rollback timer.**

```
lab@PE1# set system host-name PE1-TEST
lab@PE1# commit confirmed 2
commit confirmed will be automatically rolled back in 2 minutes unless confirmed
commit complete
lab@PE1# run show system host-name    ## now reads PE1-TEST
lab@PE1# commit                       ## confirm it, cancels the timer
```

Note: If you had lost access after the first commit, the router would have reverted itself after two minutes. Always use this for changes that could break your own management path.

4. **Undo the test change with rollback and confirm the diff first.**

```
lab@PE1# set system host-name PE1
lab@PE1# show | compare
[edit system]
-  host-name PE1-TEST;
+  host-name PE1;
lab@PE1# commit and-quit
```

Note: 'show | compare' shows candidate-versus-committed differences. Get in the habit of running it before every commit.

5. **Inspect how the MTU mandate is delivered through a group.**

```
lab@PE1> show configuration groups MTU
lab@PE1> show configuration apply-groups
```

Note: You will see 'interfaces <ge-> mtu 9192' inside the group and 'MTU' in apply-groups. One definition, applied to every ge- interface by the wildcard.*

Verification

show configuration | display set | match host-name

```
set system host-name PE1
```

What to look for: The committed host-name is PE1, not PE1-TEST. If it still reads PE1-TEST, your rollback change was never committed.

show interfaces terse | match "lo0|ge-0/0"

```
lo0.0    up  up  inet   10.20.255.1/32
ge-0/0/1.0  up  up  inet  10.20.0.1/30
ge-0/0/2.0  up  up  inet  10.20.0.17/30
```

What to look for: Loopback 10.20.255.1/32 and both transit interfaces are present and up. These are the addresses every later module depends on.

show system commit

```
0   2026-07-01 14:22:07 UTC by lab via cli
1   2026-07-01 14:19:55 UTC by lab via cli
```

What to look for: Your recent commits appear at the top. This is the audit trail; 'rollback 1' would return to the entry numbered 1.

show configuration apply-groups

```
apply-groups [ MTU ];
```

What to look for: MTU is applied. After M1 this line also lists ISIS-PARAMETERS. These groups must never be removed.

[screenshot 001: M0 verification: PE1 base state, host-name and apply-groups]

Break / Fix exercise

Scenario: A teammate was editing PE1 over the console and walked away. The lab diagram says the host-name is PE1, but you are not sure the running config matches what was intended, and you suspect uncommitted edits are sitting in the candidate.

Your task: Determine whether the committed configuration matches intent and, if there are stray uncommitted edits, restore a clean committed state without rebooting the node.

Hint: The difference between the candidate and the committed database is only visible from inside configuration mode. One command shows you the pending diff; one command discards it.

The instructor holds the fault-injection and fix commands in the answer key. Diagnose it yourself first.

Brief it back

Explain each of these aloud before moving on. If you cannot, re-read the primer and re-run the verification commands.

- Explain the difference between the candidate and the committed configuration, and exactly when the running network changes.
- What failure does 'commit confirmed' protect you against, and how does the timer work?
- How does 'apply-groups MTU' put 'mtu 9192' on every ge- interface without you configuring each interface individually?

M1 IS-IS Underlay

Estimated time: 60 min

Objective

Build the IS-IS Level-2 underlay that every SR-MPLS feature rides on top of. You will enable IS-IS on all transit links, confirm every adjacency reaches Up, read the link-state database, and prove loopback-to-loopback reachability. Without a correct underlay, nothing above it works.

Before you start: M0 complete: stage0-base committed on every node, so each transit interface already carries family inet, family iso, and family mpls, and each loopback has its ISO NET address.

Concept primer

This lab runs a single-level IS-IS domain, Level 2 only. Level 1 is disabled and 'wide-metrics-only' is set so the router advertises the modern wide (32-bit) metric TLVs that Segment Routing and Traffic Engineering require. All of this ships in the ISIS-PARAMETERS group, applied with apply-groups, which also marks every ge- interface point-to-point.

Each router's identity in IS-IS is its NET address, formatted 49.0001.<loopback-in-hex>.00. PE1's loopback 10.20.255.1 becomes 49.0001.0100.0000.0001.00. The loopback interface is placed in IS-IS as 'passive' so its /32 is advertised as a reachable prefix but no adjacency is ever attempted over it; that /32 is what SR node-SIDs and BGP next-hops resolve to.

'traffic-engineering l3-unicast-topology' and 'advertisement always' populate the TE database from the regular unicast topology and keep it advertised even without an RSVP session. Later modules (Flex-Algo, TI-LFA) read that TE database. IS-IS chooses paths by lowest cumulative metric, not hop count, so the metrics you set here decide which plane traffic takes. Every core link is metric 10; the C1-C2 cross-link is metric 30 so they are used only as backups.

Build steps

6. **Apply the ISIS-PARAMETERS group and enable the TE topology.**

```
set groups ISIS-PARAMETERS protocols isis interface <ge-*> point-to-point
set groups ISIS-PARAMETERS protocols isis level 1 disable
set groups ISIS-PARAMETERS protocols isis level 2 wide-metrics-only
set apply-groups ISIS-PARAMETERS
set protocols isis traffic-engineering l3-unicast-topology
```

```
set protocols isis traffic-engineering advertisement always
```

Note: These six lines are immutable mandates. They must appear on every PE, core, and transit node.

7. **Make the loopback passive so its /32 is advertised, not adjacent.**

```
set protocols isis interface lo0.0 passive
```

8. **Enable IS-IS on each transit interface with its metric.**

```
set protocols isis interface ge-0/0/1.0
set protocols isis interface ge-0/0/1.0 level 2 metric 10
set protocols isis interface ge-0/0/2.0
set protocols isis interface ge-0/0/2.0 level 2 metric 10
commit and-quit
```

Note: PE1's two transit links are ge-0/0/1 (toward C1, plane A) and ge-0/0/2 (toward C2, plane B). Each node's interfaces and metrics are in its own configs/stage1-isis/<node>.set. Repeat per node, or push the whole stage with 'python push_lab_configs.py stage1-isis'.

Verification

show isis adjacency

```
Interface             System   L State   Hold (secs)
ge-0/0/1.0            C1       2  Up          23
ge-0/0/2.0            C2       2  Up          19
```

What to look for: Every expected neighbor is Up. PE1 and PE2 have two neighbors each; the cores have three each. A Down or Init state points at a physical link, MTU, or family-iso problem, not a metric problem.

show isis interface

```
Interface     L  CirID Level 1 DR   Level 2 DR   L1/L2 Metric
lo0.0         0  0x1   Passive      Passive       0/0
ge-0/0/1.0    2  0x1   Disabled     Point to Point  10/10
```

What to look for: lo0.0 is Passive; transit links are Point to Point at metric 10. A wrong metric here is exactly the class of fault in this module's break/fix.

show isis database

```
IS-IS level 2 link-state database:
LSP ID        Sequence  Checksum  Lifetime
PE1.00-00     0x12      0x8a3c    1094
PE2.00-00     0x0f      0x77b1    1101
C1.00-00      0x11      0x64d2    1098
```

```
C2.00-00     0x10     0x51a9   1102
```

What to look for: All four SR nodes (PE1, PE2, C1, C2) appear in the database. A missing LSP means that node has no adjacency into the domain.

ping 10.20.255.2 source 10.20.255.1 rapid

```
PING 10.20.255.2 (10.20.255.2): 56 data bytes
!!!!!
round-trip min/avg/max = 1.2/2.0/3.1 ms
```

What to look for: PE1's loopback reaches PE2's loopback through the IS-IS underlay. This is the plain-IP reachability that M2 will move onto MPLS label-switched paths.

[screenshot 002: M1 verification: IS-IS adjacencies Up and full database]

Break / Fix exercise

Scenario: IS-IS is up and PE1 can reach PE2's loopback, but a traceroute shows traffic taking the plane-B path (PE1-C2-PE2) when the design says plane A should be equal-cost. Every adjacency reads Up, so nothing looks wrong at first glance.

Your task: Find why SPF prefers the wrong plane and restore equal-cost balance across plane A and plane B, using only IS-IS metric configuration.

Hint: Adjacency Up tells you nothing about metrics. Compare the advertised metric on each plane-A link in the database against the design value of 10, and check the total cost of 'show route 10.20.255.2' against the expected 40.

The instructor holds the fault-injection and fix commands in the answer key. Diagnose it yourself first.

Brief it back

Explain each of these aloud before moving on. If you cannot, re-read the primer and re-run the verification commands.

- Explain how IS-IS builds its SPF tree by cumulative metric rather than hop count, and how that decides plane A versus plane B in this topology.
- Why is the loopback configured as passive instead of as a normal IS-IS interface?
- What does 'traffic-engineering l3-unicast-topology' add that plain IS-IS routing does not, and which later modules depend on it?

M2 SR-MPLS Transport

Estimated time: 60 min

Objective

Turn the IS-IS underlay into a Segment Routing MPLS transport layer. You will enable SPRING (source-packet-routing) on top of IS-IS, define the shared label block, give each node a unique node-SID, and confirm that every loopback is now reachable over a label-switched path instead of plain IP. By the end PE1 can build and trace an MPLS tunnel to PE2 using label 16002, which is the transport that L3VPN services in later modules ride on.

Before you start: M1 complete: IS-IS Level 2 is up on every node, all adjacencies read Up, and loopback-to-loopback IP reachability is proven, so the SPF tree SR will reuse is already correct.

Concept primer

Segment Routing runs as an extension of IS-IS, not a separate protocol. Junos calls it SPRING, configured under 'protocols isis source-packet-routing'. There is no LDP or RSVP signalling: IS-IS floods a few new sub-TLVs in the LSPs you already have, and every router computes the same label bindings from the same link-state database. The source router decides the whole path by pushing a stack of labels called segments, which is why it is 'source' packet routing. In this lab a single node-SID per destination is enough, so each tunnel is a one-label stack.

All routers share one label block called the SRGB (Segment Routing Global Block). This lab sets 'srgb start-label 16000 index-range 8000', so the block runs from label 16000 to 23999 and is identical on every node. Each router is then given a node-segment 'ipv4-index', and its absolute node-SID label is simply SRGB-start plus that index. PE1 uses index 1, so its label is 16001; PE2 uses index 2, so its label is 16002. Because the SRGB is global and consistent, label 16002 means 'the path to PE2' from the point of view of every router in the domain. The index must be globally unique: two routers sharing an index both claim the same label, which is the fault this module's break/fix explores.

Two moving parts complete the picture. 'explicit-null' tells upstream neighbors to send the explicit-null label (label 0) on the final hop instead of popping the label entirely (penultimate-hop popping), which keeps the MPLS header on the wire all the way to the egress so QoS EXP bits survive to the last router. Two forwarding tables appear once SR is up: 'mpls.0' is the transit label table, holding the Swap

and Pop actions each router applies to labels passing through it, and 'inet.3' is the tunnel table, where each remote loopback resolves to a 'Push <node-SID>' action. BGP next-hops for L3VPN resolve out of inet.3, which is how services get placed on SR transport. Prove the data plane with a labeled 'traceroute mpls segment-routing isis', which walks the actual label-switched path rather than plain IP.

Build steps

9. **Enable SPRING and define the global label block (SRGB).**

```
set protocols isis source-packet-routing srgb start-label 16000 index-range 8000
```

Note: The SRGB is identical on every node: start 16000, range 8000, so the block is 16000 to 23999 domain-wide. This line never changes per node.

10. **Assign this node's unique node-SID index.**

```
set protocols isis source-packet-routing node-segment ipv4-index 1
```

Note: Index 1 is PE1, giving absolute label 16001 (16000 + 1). PE2 uses index 2 (label 16002), P1 index 21, P3 index 23, and so on. Each node's own index lives in its configs/stage2-sr/<node>.set; the index must be globally unique or two nodes claim the same label.

11. **Advertise explicit-null so the label survives to the egress hop.**

```
set protocols isis source-packet-routing explicit-null
```

Note: This requests the explicit-null label (0) on the final hop instead of penultimate-hop popping, keeping the MPLS header, and its EXP bits, on the wire all the way to the destination PE.

12. **Turn on MPLS forwarding and add MPLS to every transit interface plus lo0.**

```
set protocols mpls traffic-engineering mpls-forwarding
set protocols mpls interface lo0.0
set protocols mpls interface ge-0/0/1.0
set protocols mpls interface ge-0/0/2.0
commit and-quit
```

Note: family mpls was already on these interfaces from stage0-base; this stanza is what actually programs mpls.0 and lets the interface switch labels. PE1's transit links are ge-0/0/1 (plane A) and ge-0/0/2 (plane B). Each node lists its own interfaces in configs/stage2-sr/<node>.set, so repeat per node or push the whole stage with 'python push_lab_configs.py stage2-sr'. After this push, or any later push that touches segment-routing configuration, restart routing on every node in the

topology, not only the ones you changed; the SRGB does not take effect until then, and a skipped node leaves IS-IS and BGP looking perfectly healthy while forwarding is silently broken. M7's build walks the full story.

Verification

show route 10.20.255.2 table inet.3 extensive

```
inet.3: 7 destinations, 7 routes (7 active, 0 holddown, 0 hidden)
10.20.255.2/32 (1 entry, 1 announced)
      *L-ISIS Preference: 14
            Next hop: 10.20.0.2 via ge-0/0/1.0, selected
            Label operation: Push 16002
            Label TTL action: prop-ttl
```

What to look for: PE2's loopback now resolves to an SR tunnel in inet.3 with 'Push 16002'. This is the transport L3VPN next-hops resolve over; if inet.3 has no entry for 10.20.255.2, SR has not built the tunnel and no service will forward.

show route table mpls.0 | match 16002

```
16002              *[L-ISIS/14] 00:12:14, metric 30
                  >  to 10.20.0.6 via ge-0/0/1.0, Swap 16002
```

What to look for: Label 16002 (PE2's node-SID) resolves to a single next-hop with a Swap toward PE2. A single authoritative next-hop is correct. If 16002 shows two next-hops at different routers, two nodes are advertising the same index, which is a SID clash.

traceroute mpls segment-routing isis 10.20.255.2

```
Probe options: ttl 30, retries 3
 ttl  Label  Protocol   Address        Previous Hop     Probe Status
   1   16002  SR-ISIS    10.20.0.2       (null)          Success
   2   16002  SR-ISIS    10.20.0.6       10.20.0.2         Success
   3      0   SR-ISIS    10.20.0.10      10.20.0.6         Success
 FEC-Stack-Sent: SR-ISIS
 segment-routing traceroute done
```

What to look for: The labeled traceroute walks the label-switched path hop by hop, each transit router swapping 16002 until the explicit-null (0) label on the final hop, and ends in 'done'. This is the data-plane proof the SR tunnel to PE2 actually forwards, not just that a route exists.

show isis overview | match SPRING|Source

```
SPRING based MPLS: Enabled
Source Packet Routing (SPRING): Enabled
```

```
SRGB Config Range: start-label 16000 index-range 8000
Node Segments: ipv4-index 1
```

What to look for: SPRING is enabled with the shared SRGB 16000/8000 and PE1's own index 1. Confirm each node reports its own index here; a node showing the wrong or a duplicate index is the root of a label clash.

[screenshot 003: M2 verification: label-switched path PE1 to PE2]

Break / Fix exercise

Scenario: SR is enabled across the lab and IS-IS is fully up, but the transport tunnel to PE2 is misbehaving. From PE1, 'traceroute mpls segment-routing isis 10.20.255.2' either times out or returns an inconsistent hop count that never reaches PE2, even though a plain IP 'ping 10.20.255.2' still succeeds. On a transit router, 'show route table mpls.0 | match 16002' resolves PE2's node-SID label toward a next-hop that is clearly not on the path to PE2, and inet.3 on PE1 shows the 10.20.255.2 tunnel resolving to the wrong router.

Your task: The IP underlay is healthy, so the fault lives in the SR transport layer, not IS-IS. Diagnose why label 16002 is being switched toward the wrong node and restore a clean end-to-end label-switched path from PE1 to PE2, using only source-packet-routing configuration. Confirm the fix with mpls.0 showing a single correct next-hop and a labeled traceroute that completes.

Hint: Adjacency state and IP reachability tell you nothing here; work from the label tables. Label 16002 is PE2's node-SID, and its absolute value is SRGB-start plus an index, so ask which router is advertising that index. Compare each node's advertised node-segment index (visible in 'show isis overview' or the IS-IS database) against the design value for its loopback, and remember that every index in the SRGB must be globally unique.

The instructor holds the fault-injection and fix commands in the answer key. Diagnose it yourself first.

Brief it back

Explain each of these aloud before moving on. If you cannot, re-read the primer and re-run the verification commands.

- Explain how a node-SID label is derived from the SRGB and the node index, and why label 16002 means 'the path to PE2' on every router in the domain.

- What is the difference between mpls.0 and inet.3, and which one does an L3VPN BGP next-hop resolve over to get onto SR transport?
- Why does a labeled 'traceroute mpls segment-routing isis' prove something that a plain 'ping' to the same loopback does not?

M3 L3VPN Services

Estimated time: 75 min

Objective

Turn the SR-MPLS transport you built in M2 into a real customer service. You will stand up the iBGP overlay to the C1 and C2 route reflectors with family inet-vpn, build the CUST_A and CUST_B VRFs on the PEs, run eBGP to the customer edge routers, and prove that Linux Client1 behind CE1 reaches Linux Client2 behind CE2 entirely inside CUST_A. You will then prove that CUST_A and CUST_B stay completely isolated even though both customers advertise the identical 192.168.50.0/24 prefix.

Before you start: M2 complete: SR-MPLS transport is up and every remote PE loopback resolves to a Push label in inet.3, so the overlay has working tunnels to ride on.

Concept primer

An L3VPN separates two planes of routing. The transport (underlay) you already have carries packets between PE loopbacks over SR label-switched paths. The overlay is a single iBGP mesh that carries the customer routes on top of that transport. Rather than peer every PE to every other PE, each PE runs one iBGP session to C1 and one to C2, the two core route reflectors, and enables 'family inet-vpn unicast' so the session carries VPN-IPv4 (labelled L3VPN) routes. The reflectors re-advertise each PE's VPN routes to all the others, so adding a PE never touches the existing ones. The PE next-hop on every VPN route is a remote PE loopback, which resolves through inet.3 to an SR tunnel; that is the exact handoff between the overlay and the transport you built in M2.

Two BGP communities do the heavy lifting, and new engineers constantly confuse them. The route distinguisher (RD) is an 8-byte value prepended to each customer prefix to make it globally unique in the bgp.l3vpn.0 table. CUST_A on PE1 uses RD 10.20.255.1:100, so CE1's 192.168.50.1/32 is advertised as 10.20.255.1:100:192.168.50.1/32. The RD does no filtering at all; it only guarantees that CUST_A's 192.168.50.0/24 and CUST_B's identical 192.168.50.0/24 are two distinct NLRI and never overwrite each other in BGP. The route target (RT) is a separate extended community that acts as the import/export filter. CUST_A tags its routes with target:64512:100 on export and imports only that value; CUST_B uses target:64512:200. A PE installs a received VPN route into a VRF only when the

route's RT matches that VRF's import target. RD makes prefixes unique; RT decides which VRF they land in.

'vrf-table-label' tells the PE to allocate one MPLS label per VRF and to perform an IP lookup in that VRF's table after popping the label, which is what lets a single label serve every prefix in the VRF. On the customer side each VRF runs eBGP to its CE (PE AS 64512, CE AS 65001) over a VLAN subinterface of ge-0/0/3. Because the RTs differ, a route that CE1 sends into CUST_A can never leak into CUST_B, so both customers safely reuse 192.168.50.0/24. Client1 and Client2 both live in CUST_A on distinct subnets (10.50.1.0/24 and 10.50.2.0/24), so their traffic crosses the core as CUST_A VPN routes; the overlapping 192.168.50.1 loopbacks exist once in each VRF and are only reachable from inside their own VRF, which is the isolation you will demonstrate.

Build steps

13. **PE1: build the iBGP overlay to both route reflectors with the inet-vpn family.**

```
set protocols bgp group OVERLAY type internal
set protocols bgp group OVERLAY local-address 10.20.255.1
set protocols bgp group OVERLAY family inet-vpn unicast
set protocols bgp group OVERLAY family evpn signaling
set protocols bgp group OVERLAY neighbor 10.20.255.11
set protocols bgp group OVERLAY neighbor 10.20.255.12
```

Note: type internal makes this iBGP inside AS 64512. local-address ties the session to the loopback so it follows the SR transport. The two neighbors are C1 (10.20.255.11) and C2 (10.20.255.12), the route reflectors. family inet-vpn unicast is what upgrades this from a plain iBGP session to an L3VPN session.

14. **PE1: create the CUST_A VRF, its RD, RT, table label, subinterface, and eBGP session to CE1.**

```
set interfaces ge-0/0/3 vlan-tagging
set interfaces ge-0/0/3 encapsulation flexible-ethernet-services
set interfaces ge-0/0/3 unit 100 vlan-id 100
set interfaces ge-0/0/3 unit 100 family inet address 172.16.100.1/30
set routing-instances CUST_A instance-type vrf
set routing-instances CUST_A interface ge-0/0/3.100
set routing-instances CUST_A route-distinguisher 10.20.255.1:100
set routing-instances CUST_A vrf-target target:64512:100
set routing-instances CUST_A vrf-table-label
set routing-instances CUST_A protocols bgp group CE type external
```

```
set routing-instances CUST_A protocols bgp group CE peer-as 65001
set routing-instances CUST_A protocols bgp group CE neighbor 172.16.100.2
```

Note: instance-type vrf creates the CUST_A.inet.0 table. The RD 10.20.255.1:100 makes CUST_A prefixes globally unique; the RT target:64512:100 is the import/export filter. peer-as 65001 is the customer AS; the PE-CE link is eBGP over VLAN 100.

15. **PE1: create the CUST_B VRF the same way with its own RD and RT on VLAN 200.**

```
set interfaces ge-0/0/3 unit 200 vlan-id 200
set interfaces ge-0/0/3 unit 200 family inet address 172.16.101.1/30
set routing-instances CUST_B instance-type vrf
set routing-instances CUST_B interface ge-0/0/3.200
set routing-instances CUST_B route-distinguisher 10.20.255.1:200
set routing-instances CUST_B vrf-target target:64512:200
set routing-instances CUST_B vrf-table-label
set routing-instances CUST_B protocols bgp group CE type external
set routing-instances CUST_B protocols bgp group CE peer-as 65001
set routing-instances CUST_B protocols bgp group CE neighbor 172.16.101.2
commit
```

Note: CUST_B is identical in structure but RD 10.20.255.1:200 and RT target:64512:200 keep it a separate VPN. This is the deliberate second customer that reuses 192.168.50.0/24; the different RT is the only thing keeping it isolated from CUST_A.

16. **C1 (core route reflector): enable the overlay with a cluster ID and peer the PEs and the other reflector.**

```
set protocols bgp group OVERLAY type internal
set protocols bgp group OVERLAY local-address 10.20.255.11
set protocols bgp group OVERLAY family inet-vpn unicast
set protocols bgp group OVERLAY family evpn signaling
set protocols bgp group OVERLAY cluster 0.0.0.1
set protocols bgp group OVERLAY neighbor 10.20.255.1
set protocols bgp group OVERLAY neighbor 10.20.255.2
set protocols bgp group OVERLAY neighbor 10.20.255.12
commit
```

Note: cluster 0.0.0.1 makes C1 a route reflector; the PE neighbors (10.20.255.1 PE1, 10.20.255.2 PE2) become clients and their VPN routes are reflected to each other. 10.20.255.12 is C2, the redundant reflector. C2 mirrors this config with local-address 10.20.255.12. The reflectors carry no VRF; they only relay VPN NLRI.

17. **CE1 (customer edge): run eBGP up to PE1 and advertise the customer prefixes from a virtual-router instance.**

```
set routing-instances CUST_A instance-type virtual-router
set routing-instances CUST_A interface ge-0/0/1.100
set routing-instances CUST_A interface lo0.100
set interfaces lo0 unit 100 family inet address 192.168.50.1/32
set interfaces ge-0/0/2 unit 0 family inet address 10.50.1.1/24
set routing-instances CUST_A interface ge-0/0/2.0
set routing-instances CUST_A protocols bgp group TO-PE type external
set routing-instances CUST_A protocols bgp group TO-PE peer-as 64512
set routing-instances CUST_A protocols bgp group TO-PE neighbor 172.16.100.1
set routing-instances CUST_A protocols bgp group TO-PE export ADV-CUST
set routing-options autonomous-system 65001
set policy-options policy-statement ADV-CUST term lo from protocol direct
set policy-options policy-statement ADV-CUST term lo then accept
commit
```

Note: The CE has no MPLS or VPN awareness; it is a plain eBGP speaker. instance-type virtual-router gives it a private routing table. ge-0/0/2 (10.50.1.1/24) is Client1's gateway, and ADV-CUST accepts the direct routes (the 10.50.1.0/24 client subnet and the 192.168.50.1/32 loopback) so PE1 learns them over eBGP. CE1 also builds a CUST_B virtual-router on VLAN 200 the same way, reusing 192.168.50.1/32 as lo0.200.

18. **Or push the whole service tier at once.**

```
python push_lab_configs.py stage3-l3vpn
```

Note: Each node's overlay, VRF, RR, or CE lines live in its own configs/stage3-l3vpn/<node>.set. PE1/PE2 get VRFs, C1/C2 get the reflector config, CE1/CE2 get the customer edge config. Push per node from those files or run the whole stage in one shot.

Verification

show bgp summary

```
Groups: 2 Peers: 4 Down peers: 0
Table          Tot Paths  Act Paths Suppressed  History Damp State Pending
bgp.l3vpn.0           6         6         0       0        0      0
Peer           AS     InPkt  OutPkt  Flaps Last Up/Dwn State|#Active/Received/Accepted
10.20.255.11     64512       412     410     0    3:01:22 Establ
  bgp.l3vpn.0: 3/3/3/0
10.20.255.12     64512       408     407     0    3:01:19 Establ
  bgp.l3vpn.0: 3/3/3/0
172.16.100.2  65001       210     212     0    2:58:40 Establ
```

```
 CUST_A.inet.0: 2/2/2/0
172.16.101.2  65001       205     207      0    2:58:33 Establ
 CUST_B.inet.0: 1/1/1/0
```

What to look for: The two iBGP sessions to the reflectors C1 (10.20.255.11) and C2 (10.20.255.12) are Establ and feed bgp.l3vpn.0. The two eBGP sessions to CE1 (172.16.100.2 in CUST_A) and CE2's side (172.16.101.2 in CUST_B) are Establ inside their VRFs. If a reflector session is not Establ, no remote VPN routes arrive and every VRF looks empty.

show route table CUST_A.inet.0

```
CUST_A.inet.0: 5 destinations, 5 routes (5 active, 0 holddown, 0 hidden)
+ = Active Route, - = Last Active, * = Both

10.50.1.0/24    *[BGP/170] 02:58:40, localpref 100
                  AS path: 65001 I, from 172.16.100.2
                >  to 172.16.100.2 via ge-0/0/3.100
10.50.2.0/24    *[BGP/170] 02:55:10, localpref 100, from 10.20.255.2
                  AS path: 65001 I
                >  to 10.20.0.2 via ge-0/0/1.0, Push 16002, Push 299872(top)
172.16.100.0/30 *[Direct/0] 03:02:00
                >  via ge-0/0/3.100
192.168.50.1/32 *[BGP/170] 02:58:40, localpref 100
                  AS path: 65001 I, from 172.16.100.2
```

What to look for: 10.50.1.0/24 (Client1's subnet) is learned locally from CE1 over eBGP. 10.50.2.0/24 (Client2's subnet) is the remote VPN route: its next-hop is PE2 (10.20.255.2) and it carries a two-label stack, the VPN label 299872 over the SR transport label Push 16002. That outer label is the M2 tunnel doing its job under the overlay.

show route receive-protocol bgp 10.20.255.11 table bgp.l3vpn.0

```
bgp.l3vpn.0: 6 destinations, 6 routes (6 active, 0 holddown, 0 hidden)
 Prefix                  Nexthop     Lclpref  AS path
 10.20.255.2:100:10.50.2.0/24     10.20.255.2    100     65001 I
 10.20.255.2:100:192.168.50.1/32  10.20.255.2    100     65001 I
 10.20.255.2:200:192.168.50.1/32  10.20.255.2    100     65001 I
```

What to look for: These are PE2's VPN routes reflected by C1. Note how the RD prefix (10.20.255.2:100 for CUST_A, 10.20.255.2:200 for CUST_B) makes the two identical 192.168.50.1/32 prefixes distinct NLRI, so they coexist in one table. Which VRF each lands in is decided later by the RT community, not by the RD.

Client1 -> Client2 reachability, then CUST_A / CUST_B isolation

```
client1:~$ ping -c 3 10.50.2.10
PING 10.50.2.10 (10.50.2.10) 56(84) bytes of data.
64 bytes from 10.50.2.10: icmp_seq=1 ttl=61 time=3.42 ms
64 bytes from 10.50.2.10: icmp_seq=2 ttl=61 time=2.91 ms
64 bytes from 10.50.2.10: icmp_seq=3 ttl=61 time=2.88 ms
3 packets transmitted, 3 received, 0% packet loss

lab@PE1> ping routing-instance CUST_A 192.168.50.1 count 2 rapid
!!  round-trip min/avg/max = 1.9/2.1/2.3 ms   (reaches CE1 CUST_A loopback)
lab@PE1> ping routing-instance CUST_B 192.168.50.1 count 2 rapid
!!  round-trip min/avg/max = 1.8/2.0/2.2 ms   (reaches CE1 CUST_B loopback)
```

What to look for: End to end: Client1 (10.50.1.10) reaches Client2 (10.50.2.10) across the core inside CUST_A, ttl 61 showing the four router hops CE1-PE1-PE2-CE2. The two pings to the identical 192.168.50.1 both succeed but land on different loopbacks because each stays in its own VRF. Same address, two customers, zero leakage: that is the isolation the RTs enforce.

[screenshot 004: M3 verification: CUST_A client-to-client reachability and CUST_B isolation]

Break / Fix exercise

Scenario: The service is cut over and CUST_A is working: Client1 and Client2 ping each other cleanly. CUST_B is a different story. Both PEs report every BGP session Up, the SR transport is healthy, and nothing in the logs looks wrong, yet CUST_B reachability across the core fails. When you look in bgp.l3vpn.0 the remote CUST_B prefixes are clearly present and active, but they never appear in the CUST_B.inet.0 VRF table on the affected PE, so that customer's traffic has nowhere to go.

Your task: Explain why the routes can sit in bgp.l3vpn.0 while every BGP session stays Up, then find why they are not being installed into CUST_B.inet.0 and restore CUST_B reachability. Confirm CUST_A is untouched throughout, and prove the fix by installing the remote CUST_B route into the VRF and pinging across the core.

Hint: A route reaching bgp.l3vpn.0 but not the VRF is an import decision, not a session or transport problem, which is why nothing is Down. The thing that decides whether a VPN route enters a given VRF is an extended community, and it must match on both the sending and receiving side. Compare what the CUST_B routes carry in bgp.l3vpn.0 against what the CUST_B VRF is configured to import on the broken PE, and remember the healthy PEs are the reference for the correct value.

The instructor holds the fault-injection and fix commands in the answer key. Diagnose it yourself first.

Brief it back

Explain each of these aloud before moving on. If you cannot, re-read the primer and re-run the verification commands.

- Explain the difference between a route distinguisher and a route target, and describe exactly how each one contributes to letting CUST_A and CUST_B both use 192.168.50.0/24 without conflict.
- During a route-target mismatch on one VRF, why do all BGP sessions stay Up and why do the affected routes still show up in bgp.l3vpn.0, even though the VRF table and customer pings are broken?
- Trace the full control-plane path of a single CUST_A customer route from CE1 to CE2: how it enters PE1, what RD and RT and labels get attached, how the route reflectors move it, and how PE2 decides to install it in CUST_A.inet.0 and hand it to CE2.

M4 Traffic Engineering (Flexible Algorithm)

Estimated time: 75 min

Objective

Build two independent shortest-path planes out of one IS-IS topology, each computed by a different Flexible Algorithm, then steer a customer VPN onto the one you want. Flex-Algo 128 optimizes on the IGP metric and follows plane A; Flex-Algo 129 optimizes on measured link delay. You will define both algorithms, bind them to colored transport-classes so they resolve in inetcolor.0, tag CUST_B with the color-129 community so its VRF rides the delay plane, then use GNS3 to inject about 80 ms of latency on a plane-A link and watch algo-129 recompute and move CUST_B onto plane B while CUST_A on algo-128 does not move.

Before you start: M3 complete: the iBGP overlay is up through the C1 and C2 route reflectors and both L3VPN services (CUST_A target:64512:100, CUST_B target:64512:200) are established with CE reachability on every PE.

Concept primer

A Flexible Algorithm is a separate SPF calculation that runs alongside the default algo-0 shortest-path tree. Each definition names a metric type to minimize and, optionally, link constraints to include or exclude, and every router in the domain that participates advertises which algorithms it supports plus a distinct prefix-SID per algorithm for its loopback. Because each algorithm has its own SID and its own SPF result, one physical topology yields several logically distinct forwarding planes at once. In this lab algo-128 is defined with metric-type igp-metric, so it reproduces the familiar lowest-IGP-cost path and follows plane A; algo-129 is defined with metric-type delay-metric, so it minimizes cumulative measured one-way link delay instead of IGP cost.

The two planes only differ when the metrics differ. Every core link is IGP metric 10, so algo-128 picks plane A (PE1-C1-PE2) as the lowest-cost path to PE2. Algo-129 instead sums the delay values that IS-IS distributes as APP-LRTT sub-TLVs, which come from the delay-measurement probes each transit interface runs. When every link reports a low, similar delay the two algorithms agree, but the moment one plane-A link starts reporting a large delay, algo-129's SPF re-solves and prefers the lower-delay plane B while algo-128 stays put because IGP metrics have not changed. That is exactly the cause and effect you will trigger by hand later in this module.

To make traffic actually use an algorithm you connect three pieces. First, 'use-transport-class' on each flex-algo definition, plus a transport-class named for its color (color 128, color 129), causes each algorithm's SR tunnels to be installed in the colored transport RIB inetcolor.0 keyed by color rather than by plain loopback. Second, a BGP VPN route that carries an extended color community (color:0:129) is resolved through the COLOR-RESOLUTION-MAP in ip-color mode, so instead of resolving over the default inet.3 tunnel it resolves over the matching colored entry in inetcolor.0. Third, on CUST_B the import policy SET-COLOR-129 stamps every CE route with color:0:129, so the whole VRF rides algo-129 while CUST_A, which carries no color, keeps resolving over algo-128. Steering is therefore per-VRF and driven entirely by a community, with no per-flow configuration.

Build steps

19. **Define both Flexible Algorithms and their colored transport-classes.**

```
set routing-options flex-algorithm 128 definition metric-type igp-metric
set routing-options flex-algorithm 128 color 128
set routing-options flex-algorithm 128 use-transport-class
set routing-options flex-algorithm 129 definition metric-type delay-metric
set routing-options flex-algorithm 129 color 129
set routing-options flex-algorithm 129 use-transport-class
set routing-options transport-class auto-create
set routing-options transport-class name ipv4-tc-128-igp color 128
set routing-options transport-class name ipv4-tc-129-delay color 129
```

Note: algo-128 minimizes the IGP metric, algo-129 minimizes measured delay. 'use-transport-class' plus a color-matched transport-class is what lands each algorithm's tunnels in inetcolor.0 keyed by color 128 or 129. These same nine lines are identical on every SR node.

20. **Advertise participation in both algorithms in IS-IS.**

```
set protocols isis source-packet-routing flex-algorithm 128
set protocols isis source-packet-routing flex-algorithm 129
```

Note: This tells the domain that this router computes algo-128 and algo-129 and will honor their prefix-SIDs. A node that does not advertise an algorithm is pruned from that algorithm's SPF tree.

21. **Advertise a distinct prefix-SID per algorithm for this loopback.**

```
set policy-options policy-statement EXPORT-FLEX-ALGO-SID term sid from interface lo0.0
```

```
set policy-options policy-statement EXPORT-FLEX-ALGO-SID term sid from route-filter 10.20.255.1/32 exact
set policy-options policy-statement EXPORT-FLEX-ALGO-SID term sid then prefix-segment algorithm 128 index 1001
set policy-options policy-statement EXPORT-FLEX-ALGO-SID term sid then prefix-segment algorithm 128 node-segment
set policy-options policy-statement EXPORT-FLEX-ALGO-SID term sid then prefix-segment algorithm 129 index 2001
set policy-options policy-statement EXPORT-FLEX-ALGO-SID term sid then prefix-segment algorithm 129 node-segment
set policy-options policy-statement EXPORT-FLEX-ALGO-SID term sid then accept
set protocols isis export EXPORT-FLEX-ALGO-SID
```

Note: Each algorithm needs its own SID index so the two planes never share a label. The convention in this lab is algo-128 index 1000+node and algo-129 index 2000+node: PE1 (node 1) uses 1001 and 2001, P1 (node 21) uses 1021 and 2021. The absolute label is SRGB-start 16000 plus the index. Change only the two index values per node.

22. **Turn on delay measurement so algo-129 has a metric to minimize.**

```
set protocols isis interface ge-0/0/1.0 delay-measurement probe-interval 1
set protocols isis interface ge-0/0/1.0 delay-measurement probe-count 3
set protocols isis interface ge-0/0/1.0 delay-measurement advertisement periodic threshold 90
set protocols isis interface ge-0/0/1.0 delay-measurement advertisement periodic interval 60
set protocols isis interface ge-0/0/2.0 delay-measurement probe-interval 1
set protocols isis interface ge-0/0/2.0 delay-measurement probe-count 3
set protocols isis interface ge-0/0/2.0 delay-measurement advertisement periodic threshold 90
set protocols isis interface ge-0/0/2.0 delay-measurement advertisement periodic interval 60
```

Note: Each transit interface probes its link, then advertises the measured one-way delay as an IS-IS APP-LRTT sub-TLV that algo-129 sums. Without an active delay-measurement stanza on a link, algo-129 has no delay value for it and treats that link as infinite delay. Apply on every transit interface on the node: P1 has three (ge-0/0/1, ge-0/0/2, ge-0/0/3), PE1 has two.

23. **PE only: enable color resolution and steer CUST_B onto algo-129.**

```
set policy-options resolution-map COLOR-RESOLUTION-MAP mode ip-color
set policy-options policy-statement VPN-COLOR-RESOLUTION term map then resolution-map COLOR-RESOLUTION-MAP
set protocols bgp group OVERLAY import VPN-COLOR-RESOLUTION
set policy-options community COLOR-129 members color:0:129
set policy-options policy-statement SET-COLOR-129 term t1 then community add COLOR-129
set policy-options policy-statement SET-COLOR-129 term t1 then accept
set routing-instances CUST_B protocols bgp group CE import SET-COLOR-129
```

```
commit and-quit
```

Note: This block lives only on the PEs (PE1, PE2), not on P or C routers. The ip-color resolution-map on the OVERLAY session lets colored VPN routes resolve over inetcolor.0. SET-COLOR-129 tags every CUST_B CE route with color:0:129, so the whole CUST_B VRF rides algo-129. CUST_A carries no color and keeps resolving over algo-128.

24. **In GNS3, inject latency on a plane-A link (no router config).**

```
## GNS3 GUI action, not a Junos command:
## right-click the PE1 <-> P1 (plane A) link > Packet filters...
## add a 'delay' filter of ~80 ms, apply, and OK
## commit nothing on the routers: this is a GNS3 link filter
```

Note: A GNS3 per-link delay filter adds latency to the frames the IS-IS delay-measurement probes ride on, so P1 and PE1 begin advertising a much larger APP-LRTT value for that link. Algo-129 re-solves and prefers plane B; algo-128 is unchanged because the IGP metric is still 10. This is the learner-triggered cause and effect at the heart of the module. Remove the filter afterwards to restore the baseline.

Verification

show route table inetcolor.0

```
10.20.255.2-128<c>/64
                *[L-ISIS/14] 00:15:32, metric 40
                 >  to 10.20.0.2 via ge-0/0/1.0, Push 16002
10.20.255.2-129<c>/64
                *[L-ISIS/14] 00:15:32, metric 300
                 >  to 10.20.0.2 via ge-0/0/1.0, Push 16002
```

What to look for: Two colored transport routes to PE2 exist: the -128 entry (IGP metric 40) and the -129 entry (summed delay). Before latency injection both leave via the plane-A next-hop 10.20.0.2 (ge-0/0/1 toward P1). A missing -129 entry means algo-129 has no delay data on some link and has not converged.

show isis database extensive | match delay

```
IS extended reachability delay: 100 usec (ge-0/0/1.0)
IS extended reachability delay: 100 usec (ge-0/0/2.0)
 ... one advertised delay per transit link, all planes present
```

What to look for: Every transit link that runs delay-measurement advertises an APP-LRTT delay sub-TLV. All plane-A and plane-B links should show a value. A

link with no delay line is invisible to algo-129 and is treated as infinite delay, which is exactly this module's break/fix.

show route table CUST_B.inet.0 detail

```
192.168.20.0/24 (1 entry, 1 announced)
     *BGP    Preference: 170/-101
          Communities: target:64512:200 color:0:129
          Next hop type: Indirect, resolved over inetcolor.0
          Push 16002  (via the algo-129, color 129 tunnel)
```

What to look for: CUST_B's remote CE prefix carries color:0:129 and resolves over the inetcolor.0 algo-129 tunnel, not the default inet.3 tunnel. This proves the VRF is steered onto the delay plane. If Communities shows no color, SET-COLOR-129 did not run and CUST_B has fallen back to algo-128.

show route table inetcolor.0 10.20.255.2-129<c>/64 (before, then after GNS3 latency)

```
before:  *[L-ISIS/14] metric 300
          >  to 10.20.0.2 via ge-0/0/1.0, Push 16002   (plane A, toward P1)
after:   *[L-ISIS/14] metric 80300
          >  to 10.20.0.18 via ge-0/0/2.0, Push 16002  (plane B, toward P2)
```

What to look for: Once the ~80 ms GNS3 filter is applied to the PE1-P1 link, algo-129's summed delay to PE2 jumps and the -129 next-hop flips from ge-0/0/1 (plane A, P1) to ge-0/0/2 (plane B, P2). Re-run 'show route table CUST_B.inet.0' and CUST_B now follows plane B, while the -128 entry and CUST_A stay on plane A. Remove the filter to watch it flip back.

[screenshot 005: M4 verification: CUST_B follows the delay-optimized algo-129 path]

Break / Fix exercise

Scenario: CUST_A is healthy and its route in inetcolor.0 resolves normally over the algo-128 (color 128) tunnel to PE2. CUST_B, which should ride the delay-optimized algo-129 plane, is misbehaving: on PE1 the 10.20.255.2-129 transport route in inetcolor.0 is either missing or has taken a strange detour, and 'show route table CUST_B.inet.0 detail' shows CUST_B no longer following the expected delay path toward PE2. Every IS-IS adjacency is Up, every BGP session is Established, and algo-128 is completely unaffected, so nothing at the adjacency or session layer looks wrong.

Your task: Explain why only algo-129 (and therefore only CUST_B) is broken while algo-128 and CUST_A are fine, find the single transit link that is not

contributing to the delay plane, and restore normal algo-129 routing so CUST_B returns to its correct delay-optimized path. Fix only what is broken and do not touch the immutable mandates or the IGP metrics.

Hint: Algo-128 minimizes the IGP metric and algo-129 minimizes measured delay, so a fault that only hurts algo-129 has to be in the delay data, not the IGP topology. Compare the advertised delay sub-TLVs across all transit links: 'show isis database extensive | match delay' will show every link reporting a delay except one. Walk the plane-A links, find the interface whose delay measurement is not being advertised, and get it advertising again. A link with no delay value is treated as infinite delay by algo-129.

The instructor holds the fault-injection and fix commands in the answer key. Diagnose it yourself first.

Brief it back

Explain each of these aloud before moving on. If you cannot, re-read the primer and re-run the verification commands.

- Explain how a Flexible Algorithm builds a separate SPF plane from the same physical topology, and trace the full path by which a single BGP color community steers the CUST_B VRF onto algo-129 instead of algo-128.
- The delay advertisement for one plane-A link went missing, breaking algo-129 and CUST_B, yet algo-128 and CUST_A kept working perfectly. Explain precisely why the two algorithms responded differently to the same failed link.
- Describe how adding an ~80 ms GNS3 delay filter to a single link changes the IS-IS delay metric but not the IGP metric, and why that is enough to move CUST_B from plane A to plane B while leaving CUST_A where it was.

M5 Quality of Service

Estimated time: 60 min

Objective

Give SR-MPLS a working class-of-service chain so a premium flow keeps its priority end to end across the core. You will classify customer traffic by DSCP at the ingress PE, map it to a forwarding class, rewrite that intent into MPLS EXP bits on every transit link, and prove the marking survives with queue counters and a packet capture. By the end you can trace one packet from its DSCP value at the PE-CE edge to its EXP value on a core link.

Before you start: M4 complete: Flex-Algo and the L3VPN overlay are up, both Linux clients are reachable inside CUST_A (Client1 10.50.1.10 behind PE1, Client2 10.50.2.10 behind PE2), and the client image has iperf3 and tcpdump installed.

Concept primer

Class of service in this lab is a marking chain, not a bandwidth guarantee. Customer traffic arrives at the ingress PE carrying a DSCP value in its IP header. A behavior-aggregate (BA) classifier on the PE-CE subinterface reads that DSCP and assigns the packet to a forwarding class: DSCP 46 (EF, code-points 101110) becomes premium-priority, and everything else (code-points 000000) becomes best-effort. The forwarding class is the router's internal name for the service level; it is what every downstream decision keys off. Once the packet is pushed into an MPLS label stack for transport across the core, its original IP DSCP is no longer what core routers look at, so the intent has to be copied into the label header. That is the job of the EXP (also called TC) bits, three bits in every MPLS label. A rewrite rule on each transit interface writes premium-priority as EXP 5 (code point 101) and best-effort as EXP 0 (code point 000), and an EXP classifier on the far end reads those bits back into a forwarding class, so the service level rides hop by hop with the labels.

The stage5 config builds this in named pieces. 'forwarding-classes' bind queue 0 to best-effort and queue 5 to premium-priority. Two schedulers describe how each class is served: BE-SCHEDULER takes 70 percent transmit rate and buffer, PREMIUM-SCHEDULER is priority strict-high with 30 percent buffer. The scheduler-map SR-MPLS-SMAP ties each forwarding class to its scheduler, and that map is applied to every transit interface. The EXP classifier SR-MPLS-EXP-CLASS and rewrite rule SR-MPLS-EXP-REWRITE are applied to unit 0 of each transit interface so the core reads incoming EXP and re-marks outgoing EXP

consistently. The extra piece that lives only on the PE is the DSCP classifier DSCP-INGRESS, applied to the PE-CE subinterfaces ge-0/0/3 unit 100 and unit 200, which is where customer DSCP first becomes a forwarding class. Core routers such as P1 carry only the EXP classifier, rewrite, and scheduler-map because they never see customer IP headers, only labels.

Read the proof honestly. vJunos-router runs a virtualized lite-mode PFE ('set chassis fpc 0 lite-mode' in every base config), so classification, forwarding-class assignment, EXP rewrite, and queue counters are configuration-faithful and observable, but hardware-accurate scheduling, shaping, and tail-drop under real congestion are not reproduced. In plain terms: this module proves the marking and classification chain is correct (the right packets land in the right forwarding class and leave with the right EXP bits), and it proves that with queue counters and a capture. It does not prove that premium traffic would actually be protected by starving best-effort during congestion, because the lite-mode PFE does not enforce the schedulers the way silicon would. Keep that line clear when you brief: correct marking, yes; real drop behavior, no.

Build steps

25. **Define the two forwarding classes and their queues (all nodes).**

```
set class-of-service forwarding-classes queue 0 best-effort
set class-of-service forwarding-classes queue 5 premium-priority
```

Note: queue 0 is best-effort, queue 5 is premium-priority. These names are what every classifier, scheduler, and rewrite rule below refers to. The same two lines appear on PE and core nodes.

26. **Define the schedulers and bind them in a scheduler-map (all nodes).**

```
set class-of-service schedulers BE-SCHEDULER transmit-rate percent 70
set class-of-service schedulers BE-SCHEDULER buffer-size percent 70
set class-of-service schedulers PREMIUM-SCHEDULER priority strict-high
set class-of-service schedulers PREMIUM-SCHEDULER buffer-size percent 30
set class-of-service scheduler-maps SR-MPLS-SMAP forwarding-class best-effort scheduler BE-SCHEDULER
set class-of-service scheduler-maps SR-MPLS-SMAP forwarding-class premium-priority scheduler PREMIUM-SCHEDULER
```

Note: PREMIUM-SCHEDULER is priority strict-high so premium drains before best-effort. On the lite-mode PFE this describes intent faithfully but is not enforced as real congestion behavior.

27. **Define the EXP classifier and EXP rewrite rule (all nodes).**

```
set class-of-service classifiers exp SR-MPLS-EXP-CLASS forwarding-class premium-priority loss-priority low code-points 101
set class-of-service classifiers exp SR-MPLS-EXP-CLASS forwarding-class best-effort loss-priority low code-points 000
set class-of-service rewrite-rules exp SR-MPLS-EXP-REWRITE forwarding-class premium-priority loss-priority low code-point 101
set class-of-service rewrite-rules exp SR-MPLS-EXP-REWRITE forwarding-class best-effort loss-priority low code-point 000
```

Note: The classifier reads incoming EXP into a forwarding class; the rewrite writes the forwarding class back out as EXP. Premium is EXP 5 (code point 101), best-effort is EXP 0 (code point 000).

28. **Apply the scheduler-map, EXP classifier, and EXP rewrite to each transit interface. PE1 has two transit links (ge-0/0/1, ge-0/0/2); a core node like P1 has three (add ge-0/0/3).**

```
set class-of-service interfaces ge-0/0/1 scheduler-map SR-MPLS-SMAP
set class-of-service interfaces ge-0/0/1 unit 0 classifiers exp SR-MPLS-EXP-CLASS
set class-of-service interfaces ge-0/0/1 unit 0 rewrite-rules exp SR-MPLS-EXP-REWRITE
set class-of-service interfaces ge-0/0/2 scheduler-map SR-MPLS-SMAP
set class-of-service interfaces ge-0/0/2 unit 0 classifiers exp SR-MPLS-EXP-CLASS
set class-of-service interfaces ge-0/0/2 unit 0 rewrite-rules exp SR-MPLS-EXP-REWRITE
```

Note: The scheduler-map binds to the physical interface; the classifier and rewrite bind to unit 0. On P1 repeat the same three lines for ge-0/0/3 (the plane cross-link). Each node's exact interface set is in configs/stage5-qos/<node>.set.

29. **PE ONLY: add the DSCP ingress classifier and apply it to the PE-CE subinterfaces. This is where customer DSCP first becomes a forwarding class.**

```
set class-of-service classifiers dscp DSCP-INGRESS forwarding-class premium-priority loss-priority low code-points 101110
set class-of-service classifiers dscp DSCP-INGRESS forwarding-class best-effort loss-priority low code-points 000000
set class-of-service interfaces ge-0/0/3 unit 100 classifiers dscp DSCP-INGRESS
set class-of-service interfaces ge-0/0/3 unit 200 classifiers dscp DSCP-INGRESS
commit and-quit
```

Note: DSCP 101110 is 46 decimal (EF) and maps to premium-priority; 000000 is best-effort. Unit 100 is CUST_A, unit 200 is CUST_B. Core routers never get this classifier because they only see labels. Push the whole stage per node with 'python push_lab_configs.py stage5-qos'.

30. **On Client1 (Linux, 10.50.1.10 in CUST_A), generate an EF flow to Client2, then a best-effort flow, so the PE has something to classify.**

```
client1$ iperf3 -c 10.50.2.10 --tos 184 -t 20    ## --tos 184 = 0xB8 = DSCP 46 (EF)
client1$ iperf3 -c 10.50.2.10 -t 20              ## no --tos = best-effort (DSCP 0)
```

Note: --tos takes the full 8-bit ToS byte: 184 decimal is DSCP 46 in the high six bits, which the DSCP-INGRESS classifier maps to premium-priority. The plain run defaults to DSCP 0, which maps to best-effort. Run each for long enough to watch counters move. The Rocky endpoint images do not ship iperf3. Install it once per rebuild with 'sudo dnf install -y iperf3'. If dnf hangs rather than failing, see the IPv6 Router Advertisement note in the lab spec: the routers advertise themselves as IPv6 default gateways on the management segment, and dnf has no Happy Eyeballs fallback to IPv4.

31. **In GNS3, capture a core link and read the MPLS EXP bits in Wireshark while the EF flow runs.**

```
## Right-click the P1 to PE1 link (plane A) in GNS3 and choose 'Start capture'.
## Re-run the EF iperf3 from Client1 so premium packets cross the link during capture.
## Open the capture in Wireshark and expand MultiProtocol Label Switching Header.
## Read the 'MPLS Experimental Bits' (TC) field on the EF flow's frames.
```

Note: The capture is the ground-truth data-plane proof: on the EF flow the MPLS EXP field reads 5, on the best-effort flow it reads 0. This is what queue counters alone cannot show you.

Verification

show class-of-service interface ge-0/0/1

```
Physical interface: ge-0/0/1, Index: 151
  Scheduler map: SR-MPLS-SMAP, Index: 57302
  Logical interface: ge-0/0/1.0, Index: 343
  Rewrite-Output   SR-MPLS-EXP-REWRITE   exp (mpls-any)
  Classifier       SR-MPLS-EXP-CLASS     exp
```

What to look for: The physical interface shows Scheduler map SR-MPLS-SMAP, and unit 0 lists both the EXP Rewrite-Output and the EXP Classifier. A missing Rewrite-Output line here is exactly this module's break/fix: MPLS frames would leave with EXP 000.

show class-of-service rewrite-rule name SR-MPLS-EXP-REWRITE

```
Rewrite rule: SR-MPLS-EXP-REWRITE, Code point type: exp
  Forwarding class         Loss priority    Code point
  best-effort              low              000
```

```
premium-priority        low           101
```

What to look for: premium-priority at low loss priority maps to code point 101 (EXP 5) and best-effort to 000. If premium mapped to 000, every transit router would queue the premium flow as best-effort no matter what the PE classified.

show interfaces queue ge-0/0/1

```
Physical interface: ge-0/0/1, Enabled, Physical link is Up
  Interface index: 151, SNMP ifIndex: 529
```

What to look for: This is the complete, verbatim output on this vJunos lite-mode chassis: no per-queue breakdown, no 'Queue: 0' or 'Queue: 5' lines, and no Transmitted or Tail-dropped counters, confirmed empty even with '| display xml', which shows no <queue> elements at all. This is a structural property of the platform's show interfaces queue RPC, not a symptom of idle traffic, and it does not change once traffic is flowing. Because this platform does not expose per-queue counters, you cannot confirm queue placement by watching a counter climb. Infer it instead the way the checks above and below this one do: from the classifier and rewrite-rule configuration, which forwarding class each DSCP and EXP value maps to, plus a packet capture, which EXP value actually left the wire. Those two together prove placement; this command cannot.

GNS3 capture of the P1 to PE1 link opened in Wireshark

```
MultiProtocol Label Switching Header
    Label: 16002 (PE2 node-SID)
    Experimental Bits: 5      <- EF flow
    Experimental Bits: 0      <- best-effort flow
```

What to look for: The EF flow's frames carry MPLS EXP = 5 on the core link, the best-effort flow carries EXP = 0. This is the end-to-end proof that the DSCP-to-forwarding-class-to-EXP chain survived the push into the label stack. It is the one check the router CLI cannot give you on its own.

[screenshot 006: M5 verification: MPLS EXP marking on a core-link capture]

Break / Fix exercise

Scenario: Customers report that the premium (EF) flow inside CUST_A behaves no differently from best-effort once it is more than one hop into the core. Every IS-IS adjacency is Up, the L3VPN is healthy, there is no session alarm, and the ingress PE still classifies EF correctly. But a packet capture on a core link shows the premium flow leaving one transit interface with MPLS EXP set to 000 instead of

101, and from that hop onward downstream routers queue everything as best-effort.

Your task: Find the single transit interface that is no longer marking premium traffic into EXP 5 and restore correct EXP marking across the core, using only class-of-service configuration. Confirm the fix in a capture, not just in the CLI.

Hint: The forwarding-class assignment is fine; what is missing is the step that copies that assignment into the MPLS label header on egress. Compare 'show class-of-service interface' on each transit interface against a known-good one and look for the object that is present on the healthy interface but absent on the faulty one. Queue counters will not flag this, because queue placement is set at ingress, not by the egress marking; only the capture and the interface's applied objects reveal it.

The instructor holds the fault-injection and fix commands in the answer key. Diagnose it yourself first.

Brief it back

Explain each of these aloud before moving on. If you cannot, re-read the primer and re-run the verification commands.

- Trace one EF packet from Client1 to Client2: name the DSCP value it carries at the PE-CE edge, the forwarding class it is assigned, and the MPLS EXP value it carries on a core link, and say which config object performs each of those steps.
- Explain why a missing EXP rewrite on just one transit link collapses service differentiation for every router downstream of that link, even though the ingress PE classified the traffic correctly and no session alarms fire.
- State precisely what the vJunos lite-mode PFE does and does not prove in this module: which parts of the class-of-service chain are configuration-faithful and observable, and which behavior (real scheduling, shaping, and tail-drop under congestion) it cannot reproduce.

M6 Fast Reroute (TI-LFA)

Estimated time: 45 min

Objective

Give the SR-MPLS core sub-second protection against a single link or node failure using Topology-Independent Loop-Free Alternate (TI-LFA). You will confirm that IS-IS has already pre-computed a repair path for PE2's loopback, read the backup next-hop in both the routing table and the forwarding table, then fail PE1's plane-A primary link and watch traffic move onto the pre-programmed backup without waiting for IS-IS to re-converge. By the end you can prove protection is armed before a failure, not computed after one.

Before you start: M5 complete and the full SR-MPLS stack running: IS-IS Level-2 underlay, SR-MPLS transport with node-SIDs, the L3VPN overlay, Flex-Algo, and class-of-service all committed on every node so that 10.20.255.2 already resolves through an SR label-switched path.

Concept primer

TI-LFA is IS-IS fast reroute for a Segment Routing core. For every protected prefix, IS-IS runs a second SPF that removes the protected link (or node) from the topology and computes where traffic would flow once the network had fully re-converged. It then pre-installs that repair path in the forwarding table as a backup next-hop, expressed as an SR label stack. The key idea is that the backup is computed and programmed into hardware ahead of time, so when a link fails the line card simply switches to the already-present backup entry. Convergence is bounded by failure detection plus a hardware next-hop swap, which is why it is sub-second rather than the seconds a full IGP re-computation would take.

The repair path is built from SR segments, which is what makes it topology-independent. A classic LFA can only protect a prefix when a directly connected neighbor happens to have a loop-free path to the destination. TI-LFA instead encodes the post-convergence path as a stack of node and adjacency SIDs, steering the repaired packet along the exact path the network will use after re-convergence, even when that path is several hops away and no natural LFA exists. In this lab the label stack resolves toward PE2's node-SID 16002 over the surviving plane, so the repaired traffic follows the same forwarding logic as the primary, just on the other path.

This behaviour is switched on by three lines that ship inside the ISIS-PARAMETERS group: 'interface <ge-*> level 2 post-convergence-lfa' arms TI-LFA on every ge- interface at once through the wildcard, 'backup-spf-options use-post-convergence-lfa' tells the backup SPF to prefer the post-convergence path, and 'backup-spf-options use-source-packet-routing' lets that backup be expressed as an SR label stack. Because they live in the group and are pulled in by apply-groups, one definition protects the whole fabric. They are immutable mandates and must never be negated. The other half of the design is physical: the P1-P2, P3-P4, and C1-C2 cross-links, all at metric 30, guarantee that when a primary plane-A link drops there is always a loop-free post-convergence path back onto plane B for the backup SPF to find. Without those cross-links TI-LFA could compute no valid repair for some prefixes, so the topology and the mandate work together.

Build steps

32. **Confirm the three TI-LFA mandates are present in the ISIS-PARAMETERS group.**

```
set groups ISIS-PARAMETERS protocols isis interface <ge-*> level 2 post-convergence-lfa
set groups ISIS-PARAMETERS protocols isis backup-spf-options use-post-convergence-lfa
set groups ISIS-PARAMETERS protocols isis backup-spf-options use-source-packet-routing
```

Note: These three lines are immutable mandates and are already present in the stage6-frr solution config on every SR node. The wildcard <ge-> arms post-convergence-lfa on all ge- interfaces at once, and the two backup-spf-options lines make the backup a post-convergence SR label stack. Do not negate or deactivate them. If they are missing, add them and 'commit'.*

33. **Verify a backup next-hop for PE2 is already pre-computed before any failure.**

```
lab@PE1> show isis route 10.20.255.2 extensive    ## look for TILFA computed MPLS paths Valid:1
lab@PE1> show route 10.20.255.2 detail            ## look for a Backup next-hop under the primary
```

Note: Protection must be armed with the network healthy. Both plane-A (ge-0/0/1 toward P1) and plane-B (ge-0/0/2 toward P2) primaries are present, and the TI-LFA backup is pre-installed. If there is no Backup next-hop now, the demonstration below will drop traffic.

34. **Fail the plane-A primary link on PE1 and observe the backup take over.**

```
lab@PE1> configure
lab@PE1# set interfaces ge-0/0/1 disable
lab@PE1# commit and-quit    ## or, in GNS3, right-click the PE1-P1 link and Stop it
```

```
lab@PE1> show route 10.20.255.2 detail    ## traffic now on the former backup path via ge-0/0/2
```

Note: Disabling ge-0/0/1 (or stopping the PE1-P1 link in GNS3) fails the plane-A primary. Because the backup was pre-programmed, forwarding moves onto plane B in sub-second time rather than after a full IGP re-convergence. Watch a running 'ping 10.20.255.2 source 10.20.255.1 rapid' lose at most a packet or two.

35. **Restore the primary link and confirm protection re-arms.**

```
lab@PE1> configure
lab@PE1# delete interfaces ge-0/0/1 disable
lab@PE1# commit and-quit    ## or Start the PE1-P1 link in GNS3
lab@PE1> show route 10.20.255.2 detail    ## primary via ge-0/0/1 with the Backup next-hop restored
```

Note: Always return the lab to full protection. Each node's stage6-frr config carries the same three group lines, so a fabric-wide reset is 'python push_lab_configs.py stage6-frr'. PE1 and P1 are shown as representatives; the mandate is identical on every SR node.

Verification

show isis route 10.20.255.2 extensive

```
10.20.255.2/32  L2  metric 40
 ge-0/0/1.0  IPV4 P1
 ge-0/0/2.0  IPV4 P2
 TILFA computed MPLS paths, Count:1, Valid:1
    PathId:0, SID count:1
```

What to look for: 'TILFA computed MPLS paths' with 'Valid:1' is the proof that IS-IS has pre-computed a usable post-convergence backup toward PE2's node-SID 16002. 'Valid:0' means the use-post-convergence-lfa mandate is missing or deactivated and no backup exists.

show route 10.20.255.2 detail

```
10.20.255.2/32 (1 entry, 1 announced)
      *IS-IS Preference: 18
       Next hop: 10.20.0.2 via ge-0/0/1.0, selected   ## primary, plane A via P1
       Next hop: 10.20.0.18 via ge-0/0/2.0          ## Backup next-hop, plane B via P2
         Label operation: Push 16002
       Backup next hop found by TI-LFA
```

What to look for: The selected primary points out ge-0/0/1 toward P1 on plane A, and a second Backup next-hop out ge-0/0/2 toward P2 on plane B is listed beneath it. A primary with no Backup next-hop means protection is not armed.

show route forwarding-table destination 10.20.255.2

```
Destination        Type RtRef Next hop           Type Index
10.20.255.2/32     user     0                    ulst
                             10.20.0.2  ge-0/0/1.0  ucst   (primary)
                             10.20.0.18 ge-0/0/2.0  ucst   (backup)
```

What to look for: The backup is programmed into the FIB itself, not just the control plane. Seeing both next-hops in the forwarding-table is what makes the failover a hardware swap and therefore sub-second. If only the primary appears here, the line card has nothing to fail over to.

show route 10.20.255.2 detail ## run again after 'set interfaces ge-0/0/1 disable ; commit'

```
10.20.255.2/32 (1 entry, 1 announced)
     *IS-IS Preference: 18
      Next hop: 10.20.0.18 via ge-0/0/2.0, selected   ## now primary, plane B via P2
       Label operation: Push 16002
```

What to look for: After the plane-A link is disabled, the former backup out ge-0/0/2 is now the selected primary and traffic to PE2 continues on plane B. The switchover happened before IS-IS ran a fresh SPF, which is the whole point of pre-computing the repair path. Re-enable ge-0/0/1 when done.

[screenshot 007: M6 verification: TI-LFA precomputed backup next-hop for PE2]

Break / Fix exercise

Scenario: The lab looks healthy and 10.20.255.2 is reachable, but during a maintenance test the team disabled PE1's plane-A link and watched a steady ping to PE2 drop packets for several seconds before recovering, instead of the sub-second blip the design promises. Every adjacency is Up and the SR label-switched path to PE2 works, so nothing looks wrong until a link actually fails.

Your task: Explain why traffic was unprotected and restore sub-second fast reroute for PE2's loopback across the fabric. Confirm your fix by proving a backup next-hop is pre-computed again before the next failure test.

Hint: With the network healthy, check 'show isis route 10.20.255.2 extensive' for 'TILFA computed MPLS paths' and its Valid count, and 'show route 10.20.255.2 detail' for a Backup next-hop. A Valid:0 result and a missing backup, with the underlying stanza still visible in the config, is the signature of a group-level mandate that has been negated rather than removed. This is a mandate-negation fault; the correct fix restores the immutable TI-LFA mandate in the ISIS-PARAMETERS group and re-arms protection on every IS-IS interface at once. Compare 'show

configuration groups ISIS-PARAMETERS | display inheritance' against the stage6-frr solution to spot which mandate is inactive.

The instructor holds the fault-injection and fix commands in the answer key. Diagnose it yourself first.

Brief it back

Explain each of these aloud before moving on. If you cannot, re-read the primer and re-run the verification commands.

- Explain how TI-LFA pre-computes a loop-free backup ahead of any failure and how it encodes that repair path as an SR label stack toward the destination's node-SID.
- Why does deactivating one line in the ISIS-PARAMETERS group remove protection on every IS-IS interface in the fabric at the same time, and why is that so easy to miss in a group-heavy config?
- Describe the role of the metric-30 cross-links (P1-P2, P3-P4, C1-C2) in guaranteeing that the backup SPF can always find a loop-free post-convergence path for the repair.

M7 End-to-End QoS Marking

Estimated time: 60 min

Objective

Follow one packet from a setsockopt() call in application source code, through the IPv4 TOS byte, across the customer LAN, into a provider edge that decides whether to believe the marking, down into three bits of MPLS EXP across the core, and back out the far side into a DSCP the receiving application reads off its own socket. By the end you can explain why sixty-four DSCP values cannot survive a network that carries eight, and why an application does not get to decide its own priority.

Before you start: M5 complete, so the class-of-service framework and the EXP rewrite are already understood, and the full SR-MPLS stack running. Both Linux endpoints reachable with the qos-e2e application tree deployed to /home/rocky/qos-e2e (see qos-e2e/deploy_clients.py).

Concept primer

An application marks itself with one line: setsockopt(IPPROTO_IP, IP_TOS, dscp << 2). That value lands in the IPv4 header's TOS byte, nowhere else. It does not touch the transport layer; ports and protocol are separate fields the network can match on independently. The shift by two is because RFC 2474 redefined the byte so DSCP owns the top six bits and ECN owns the bottom two. DSCP is not married to TOS, it is part of it.

Six bits hold sixty-four values. The MPLS EXP field, also called Traffic Class, holds three bits and therefore eight. When the ingress PE imposes a label stack it must collapse whatever DSCP arrived into one of a handful of forwarding classes, and only that class crosses the core. Every transit router in this lab reads the label and never looks at the IP header at all. On the far side the egress PE rebuilds a DSCP from the forwarding class, which means it cannot recover which of several DSCPs originally mapped into that class. The round trip is lossy, and this module makes you watch it happen.

There are two kinds of classifier and the difference is the whole trust conversation. A behaviour-aggregate (BA) classifier maps the DSCP that arrives to a forwarding class: it believes the customer completely. A multifield (MF) classifier is a firewall filter that assigns a forwarding class based on the flow's identity, five-tuple and all, regardless of what the packet claimed. Applied in the

input direction, its forwarding-class action overrides the BA result. That override is where 'my application marks itself important' stops being true.

One more thing worth knowing before you touch a router: this lab's PFE is software-simulated (vJunos in lite-mode), and it turns out to genuinely differ from hardware in two ways that matter for this module. It does not expose per-queue counters, so you cannot watch a Transmitted count climb to confirm placement, and its egress shaper does not actually enforce a rate under load even though the control plane accepts and recomputes against it. Both limitations are covered where they come up below, with the true commands to use instead.

Build steps

36. **Read the marking line in the application source. Every sender uses the same helper, so this one line is the entire claim an application can make about its own importance.**

```
client1$ grep -A8 'def set_dscp' ~/qos-e2e/isr_apps/common.py
client1$ grep -n 'set_dscp' ~/qos-e2e/isr_apps/*.py
```

Note: Four different applications, one identical line. Show this to a software audience before anything else: it is the only part of the chain they control, and it is smaller than they expect.

37. **Prove the TOS byte actually reaches the wire.**

```
client1$ sudo python3 ~/qos-e2e/isr_apps/wire_peek.py --iface eth0 --count 5 --port 9100 &
client1$ python3 ~/qos-e2e/isr_apps/c2_messaging.py --duration 5
```

Note: wire_peek prints 'CS6 (48)' where tcpdump prints 'tos 0xc0'. Same information, readable by an audience that does not think in hex. This is the last point in the chain where the application's wish and reality are guaranteed to agree.

38. **Push stage 7 and read the two classifiers side by side.**

```
python push_lab_configs.py stage7-qos-e2e
show class-of-service classifier name DSCP-TRUSTED
show configuration firewall family inet filter UNTRUSTED-REMARK
show class-of-service interface ge-0/0/3
```

Note: DSCP-TRUSTED maps six customer DSCPs onto four forwarding classes. UNTRUSTED-REMARK matches TCP port 9300 and forces best-effort with 'then dscp cs1', overriding whatever the BA classifier decided. Both are bound to ge-0/0/3.100. One trusts, one does not.

39. **CRITICAL: restart routing on every node in the topology, not only the ones this stage touched.**

```
## On EVERY router: PE1, PE2, P1, P2, P3, P4, C1, C2
restart routing
```

Note: This is the single most expensive lesson this lab build produced. After any stage push that changes segment-routing configuration, the SRGB does not take effect until routing is restarted, and it must be restarted on every node, not just the ones you edited. Skip one node and the lab looks completely healthy: IS-IS adjacencies Up, BGP established, no alarms, while forwarding is silently dead and every VPN route sits with an unresolvable next hop. This cost this build several hours to diagnose. If the next step does not work, check this before you doubt the classifier configuration.

40. **Run the console on Client2, then all four applications on Client1 with the imagery push in greedy mode.**

```
client2$ python3 ~/qos-e2e/console/render_tui.py
client1$ cd ~/qos-e2e/isr_apps && python3 c2_messaging.py --duration 45 & python3 track_feed.py --duration 45 & python3 fmv_stream.py --duration 45 & python3 imagery_push.py --greedy --duration 45 & wait
```

Note: The console reads each arriving datagram's TOS byte back off its own socket with IP_RECVTOS and compares it against what the sender recorded in the payload header. Watch the VERDICT column. Two rows are the point of the whole module. That IP_RECVTOS path only applies to the three UDP rows; imagery_push is TCP, and this lab's kernel never delivers IP_RECVTOS ancillary data for a stream socket, so its arrived DSCP is read instead with getsockopt(IPPROTO_IP, IP_PKTOPTIONS, ...), the console's TCP-specific fallback.

41. **Capture a core link and read the EXP bits for all four streams.**

```
## Right-click the PE1 to P1 link in GNS3, choose 'Start capture'.
## Re-run the senders, then open the capture in Wireshark.
## Expand MultiProtocol Label Switching Header for each stream.
```

Note: The FMV stream and the track feed are indistinguishable here. They chose different DSCPs and both carry EXP 100, because three bits cannot hold sixty-four values. This is the collapse on the wire, and it is the check the router CLI alone cannot give you. Do not expect 'monitor traffic interface ge-0/0/1' on the router itself to show any of this: monitor traffic is a routing-engine-side capture and only ever sees control-plane packets addressed to the RE, never transit customer traffic. Four attempts across both core uplinks with tens of thousands of customer packets in

flight captured none of them. The GNS3 GUI capture above is the only way to see EXP for customer data traffic in this lab, so do not lose an hour rediscovering that on your own.

42. **Toggle the egress rewrite on PE2 to move between the RFC 3270 tunnel models and watch the delivered DSCP change.**

```
## On PE2:
delete class-of-service interfaces ge-0/0/3 unit 100 rewrite-rules dscp EGRESS-DSCP
commit
## Re-run track_feed and re-read the console, then restore:
rollback 1
commit
```

Note: With the rewrite applied you are in the uniform model: the delivered DSCP is rebuilt from the forwarding class, so AF31 comes out as AF41. With it removed you are closer to the pipe model: the customer's original DSCP survives untouched end to end. Same network, different contract with the customer, one config line.

43. **Push a bottleneck onto a core link and find out whether marking buys any protection under real congestion.**

```
## GNS3 2.2.59 has no bandwidth filter; the only link filters available are frequency_drop,
packet_loss, delay (latency and jitter), corrupt, and bpf.
## Right-click the PE1 to P1 link, Packet filters:
##   Packet loss 5 %, Latency 80 ms, Jitter 20 ms, then Apply
## Watch the console loss and jitter columns for 30 seconds.
## Clear the filters and confirm recovery.
```

Note: GNS3 link filters are DSCP-blind: they degrade every packet on the link equally, so they can prove the network got worse but never that premium traffic was protected while bulk traffic was not. The real answer to that question was measured separately with class-of-service shaping (qos-e2e/spike_scheduling.md): 'shaping-rate 5m' committed cleanly on PE1's core interface, Junos even recomputed the scheduler's bandwidth math against it, and two competing flows, one EF-marked and one unmarked, were then offered at 20 Mbps each, 40 Mbps combined into that 5 Mbps ceiling. Both flows sustained their full offered rate through the shaper for the whole 30-second test with zero percent loss. The control plane confirmed a policy the forwarding plane never applied. This lab does not demonstrate protection under congestion, and it should not be shown as if it does. That gap is exactly why you test forwarding behaviour rather than trust 'show' output, and it is a more honest lesson for a software audience than a scripted demo that quietly cannot fail.

Verification

The console ledger on Client2

```
STREAM        ASKED       ARRIVED      PACKETS    LOSS    JITTER  VERDICT
-------------------------------------------------------------------------------
c2_messaging   CS6 (48)    CS6 (48)          90    0.0%     4.8ms  honored
fmv_stream     AF41 (34)   AF41 (34)       4493    0.2%     5.8ms  honored
imagery_push   EF (46)     CS1 (8)        60313     n/a      n/a  RE-MARKED AT GATEWAY
track_feed     AF31 (26)   AF41 (34)        899    0.1%     4.7ms  CLASS COLLAPSED
```

What to look for: This is the real console output this lab produced (re-measured 2026-08-06 after commit 52daca5's TCP reassembly fix, with the column header exactly as render_tui.render() emits it: ARRIVED, not DELIVERED), not an approximation. imagery_push's PACKETS column (60313) matches the sender's own reported chunk count exactly on this run; that agreement is the evidence that TCP reassembly is working, not merely plausible. c2_messaging and fmv_stream were both delivered exactly what they asked for: honored. track_feed asked for AF31 and was delivered AF41: CLASS COLLAPSED, and nothing is broken. AF31 and AF41 both collapse into assured-forwarding at the ingress PE, only the class crosses the core, and the egress rewrite rebuilds a DSCP from the class alone with no way to know which of the two original values went in. imagery_push claimed EF, a real class in this network with genuine strict-high scheduling, and was delivered CS1 anyway: RE-MARKED AT GATEWAY, because the untrusted-remark filter classifies on flow identity rather than on the claim. Its loss and jitter columns read n/a deliberately: imagery_push runs over TCP, and TCP does not lose data, it retransmits, so a loss percentage there would describe retransmissions rather than delivery and would be meaningless next to the UDP-style figures on the other three rows. That distinction is worth making to a software audience on its own.

show class-of-service interface ge-0/0/3 (on PE1)

```
Logical interface: ge-0/0/3.100
  Classifier   DSCP-TRUSTED   dscp
```

What to look for: The BA classifier is bound to the customer-facing unit. If this is missing, every customer DSCP falls to best-effort and all four streams receive identical treatment no matter what they marked. This is also the check to lean on instead of queue counters: this platform's 'show interfaces queue' returns no per-queue data at all (see M5's verify section for the full explanation), so classifier and rewrite-rule configuration plus a packet capture are the only way to confirm where a packet actually landed.

python qos_trace.py track_feed

```
1  application asks       Client1          AF31 (26)
4  gateway decides        PE1 ge-0/0/3.100   BA=on MF=on -> assured-forwarding
5  collapse into label    PE1 core egress    EXP 100
7  reverse translation    PE2 ge-0/0/3.100   rewrite=on -> DSCP 34
```

What to look for: The whole chain on one screen. Stage 4 shows whether both classifiers are in force, and stage 7 shows the delivered DSCP differing from the one the application asked for.

[screenshot 008: M7 verification: the console ledger showing collapse and re-mark]

Break / Fix exercise

Scenario: An audit reports that a bulk imagery transfer is receiving Expedited Forwarding treatment end to end, ahead of command and control traffic. Every IS-IS adjacency is Up, the L3VPN is healthy, the core is marking EXP faithfully, and the DSCP classifier on the ingress PE is present and correct. The application team confirms their transfer does set EF on its socket, and insists this is legitimate because the classifier accepts the value.

Your task: Restore correct treatment so the bulk transfer is carried as best effort regardless of what it marks, without changing the behaviour of any other stream and without asking the application team to change their code. Prove the fix from the receiving application's own output, not only from the router CLI.

Hint: A behaviour-aggregate classifier is not a security control. It maps whatever DSCP arrives to a forwarding class and believes the customer completely, which is exactly what it is designed to do. Something else has to disbelieve them. That something acts on the flow's identity rather than its marking, and it must be applied in the input direction on the customer-facing logical unit to take effect before classification. Compare 'show class-of-service interface ge-0/0/3' and 'show configuration interfaces ge-0/0/3 unit 100' on the box against the generated configs/stage7-qos-e2e/PE1.set and look for the object that exists in the file but not on the router. Queue counters will not flag this on their own, because this platform does not expose per-queue counters at all, and even where placement can be shown, placement follows classification, and the classification is doing exactly what it was told.

The instructor holds the fault-injection and fix commands in the answer key. Diagnose it yourself first.

Brief it back

Explain each of these aloud before moving on. If you cannot, re-read the primer and re-run the verification commands.

- Trace one track_feed packet end to end. Name the DSCP it carries leaving the application, the forwarding class it is assigned at the ingress PE, the EXP value it carries on a core link, and the DSCP it is delivered with. Then explain why the first and the last of those differ, without using the word 'bug'.
- Explain the difference between a behaviour-aggregate classifier and a multifield classifier. Say which one enforces a trust boundary, why the other one cannot, and what would happen if you deployed only the first.
- A software team asks you to guarantee their application priority by having it mark itself EF. Give them a complete answer: what marking does and does not achieve, where the decision actually gets made, how many distinct treatments the core can express, and what they would need to agree with the network team for their marking to mean anything at all.

M8 Capstone Break/Fix and Brief

Estimated time: 90 min

Objective

Restore a network carrying three simultaneous faults across the underlay, the SR transport, and the L3VPN service, then explain what happened. You will apply the eight-step troubleshooting flow end to end, working strictly from the lowest layer upward, prove that full customer reachability is back, and deliver a five-minute root-cause brief. This capstone tests diagnosis and communication, not your ability to type new configuration.

Before you start: M0 through M7 are complete and previously verified on this pod, and the instructor has loaded the capstone fault set so that service is currently down before you begin.

Concept primer

SR-MPLS is a strictly layered stack, and every layer depends on the one below it. IS-IS forms the underlay and runs SPF, which decides both the IGP next-hop and the set of node-SIDs each router can reach. Flex-Algo and the colored transport planes ride on that SPF result. The MPLS forwarding table and inet.3 tunnels are programmed from those SIDs, and only then does the BGP L3VPN service resolve its VPN routes onto the transport. Because each layer is computed from the one under it, a fault low in the stack does not announce itself politely at the top. It reshapes the path or the label table silently, so the service simply fails while the real cause sits two layers down.

That is why the discipline is underlay before transport before service. A broken metric moves the SPF path, which drags every SR label-switched path with it, so a transport fault on the newly chosen path can look like a healthy underlay hiding a service problem. A label clash on the active plane breaks MPLS forwarding even after the metric is corrected, so fixing the underlay alone is not enough. And a VRF import mismatch cannot be judged at all while the transport under it is broken, because the routes never had a tunnel to resolve onto. The eight steps in the troubleshooting guide are ordered exactly for this reason: confirm IS-IS adjacencies, then the algo and colored planes, then VRF steering, then CoS mapping and EXP rewrite, then the IS-IS route and TI-LFA, then the mpls.0 transit labels, then the inet.3 tunnel resolution. You always fix the lowest broken layer first, then re-run the checks above it, because clearing the lower fault often changes what the higher checks report.

A good root-cause brief is short, ordered, and honest about method. It names each fault in plain terms, states the single command output that exposed it, and explains why the layer order mattered: which lower fault was masking which higher one, and why you could not have trusted the service-layer evidence until the transport under it was clean. It closes with the customer impact and the confirmation that reachability is restored. The goal is that a listener who never saw your terminal can repeat your reasoning and reach the same conclusion.

Build steps

44. **Step 1. Confirm the IS-IS underlay adjacencies are all Up.**

```
show isis adjacency
```

Note: On every router each expected neighbor must read State Up. PE1 and PE2 have two plane neighbors each; P and C routers have two or three. If any adjacency is Down or Init, stop and fix that first, because with a missing adjacency no SIDs are distributed and every check above is unreliable. Adjacencies Up does not clear the underlay: a wrong metric still shows every neighbor Up.

45. **Step 2. Check the Flex-Algo and colored transport planes in inetcolor.0.**

```
show route table inetcolor.0
```

Note: For each remote PE loopback expect two colored entries, one ending in -128 (IGP metric, algo 128) and one ending in -129 (delay, algo 129), each pushing the remote node-SID. From PE1 both 10.20.255.2-128<c>/64 and 10.20.255.2-129<c>/64 should be present. If a plane looks wrong here, suspect the SPF result below it before blaming Flex-Algo itself.

46. **Step 3. Check the VRF service steering. Defer judgement until the layers below are clean.**

```
show route table CUST_A.inet.0
show route receive-protocol bgp 10.20.255.11 table bgp.l3vpn.0
```

Note: A clean result is the remote CE prefixes present in CUST_A.inet.0. If the VRF table is empty while the routes are clearly present in bgp.l3vpn.0, that points at an import route-target mismatch. Do not trust this reading until the transport under it is proven healthy; a broken tunnel produces the same empty table for a completely different reason.

47. **Step 4. Check the CoS interface mapping on transit links.**

```
show class-of-service interface ge-0/0/1
```

Note: The physical interface must show Scheduler map SR-MPLS-SMAP, and the .0 unit must list Rewrite-Output SR-MPLS-EXP-REWRITE and Classifier SR-MPLS-EXP-CLASS. A missing rewrite collapses QoS from that hop onward but does not itself drop reachability.

48. **Step 5. Check the EXP rewrite rule marks the right code points.**

```
show class-of-service rewrite-rule name SR-MPLS-EXP-REWRITE
```

Note: premium-priority at low loss must map to code point 101 and best-effort to 000. If premium-priority maps to 000, CUST_B traffic lands in the best-effort queue at every transit hop regardless of PE classification.

49. **Step 6. Check the IS-IS route to the remote SID and its TI-LFA backup.**

```
show isis route 10.20.255.2 extensive
show isis database
```

Note: From PE1 the route to PE2 should show the expected primary paths toward SID 16002 and TILFA computed MPLS paths with Valid: 1. This is where an underlay metric fault becomes visible: compare the advertised link metrics in the database against the design and check the total cost of show route 10.20.255.2 against the expected 40. This is the lowest layer where the capstone can bite, so fix anything wrong here before looking higher.

50. **Step 7. Check the mpls.0 transit label table for a clean SID resolution.**

```
show route table mpls.0 | match 16002
```

Note: Each node-SID must resolve to a single authoritative next-hop toward its owner. From PE1 or a transit router, label 16002 must show one Swap entry heading toward PE2. If the same label resolves toward two different routers, a duplicate SID index is present and MPLS forwarding on that plane is broken. This fault only becomes cleanly observable once Step 6 has put traffic back on the correct path, so re-run this check after any underlay fix.

51. **Step 8. Confirm end-to-end tunnel resolution in inet.3.**

```
show route 10.20.255.2 table inet.3 extensive
```

Note: From PE1 the entry for PE2 must show Label operation: Push 16002 toward the correct next-hop. If inet.3 has no entry or the push is wrong, all VRF routes stay stranded in bgp.l3vpn.0 no matter how correct the route-targets are. Only when Steps 6, 7, and 8 are clean should you go back to Step 3 and judge the service layer.

Verification

show isis adjacency

```
Interface             System   L State   Hold (secs)
ge-0/0/1.0            C1       2  Up           23
ge-0/0/2.0            C2       2  Up           19
```

What to look for: Run on each router. Every expected neighbor is Up across both planes, confirming the underlay is whole and metrics are back to the design so SPF prefers the intended paths.

traceroute mpls segment-routing isis 10.20.255.2

```
Probe options: retries 3, exp 7
 ttl  Label  Protocol   Address       Previous Hop   Probe Status
   1  16002  L-ISIS     10.20.0.2     (null)         Success
 Path 1 via ge-0/0/1.0 destination 10.20.255.2
 mpls-traceroute completed
```

What to look for: From PE1 the SR label-switched path to PE2 completes end to end with label 16002. This proves the transport layer is healthy: the metric path is correct and the SID resolves to one owner with no clash.

show route table CUST_A.inet.0

```
CUST_A.inet.0: 6 destinations, 6 routes (6 active, 0 holddown)
192.168.10.0/24    *[BGP/170] 00:02:11, localpref 100
                      AS path: 65001 I, Push 16002
10.50.2.0/24       *[BGP/170] 00:02:11, localpref 100
```

What to look for: Run on PE2. The CE prefixes learned across the core are present in the CUST_A VRF, which confirms the VPN import route-target now matches target:64512:100 and the service layer has recovered on top of a clean transport.

ping 10.50.2.10 source 10.50.1.10 rapid ## from Client1 to Client2

```
PING 10.50.2.10 (10.50.2.10): 56 data bytes
!!!!!
round-trip min/avg/max = 2.1/3.4/5.0 ms
```

What to look for: The end-to-end customer test from Client1 (10.50.1.10) to Client2 (10.50.2.10) succeeds. All three layers are restored and full CUST_A service is proven from the customer edge, not just from the PE.

[screenshot 009: M8 verification: full service restored and root-cause brief delivered]

Break / Fix exercise

Scenario: The instructor has loaded the capstone fault set and customer service is down. Three faults are active at the same time, one in the IS-IS underlay, one in the SR transport, and one in the L3VPN service. From the top the symptom looks like a single service outage, but the real cause is spread across all three layers, and the lower faults are changing what the higher checks report.

Your task: Diagnose in strict layer order using the eight-step flow: prove the underlay clean, then the transport, then the service, fixing the lowest broken layer first and re-checking upward after each fix. Restore full reachability from Client1 to Client2, then deliver a five-minute root-cause brief that names each fault, states the one command output that exposed it, and explains why the diagnosis order mattered.

Hint: Resist the urge to jump straight to the service layer just because the service is what failed. A metric fault or a label clash below will mask or mimic a VRF fault above, so an empty VRF table can be lying to you. Verify each layer is genuinely clean before you move up, and re-run the checks above a layer after you fix it, because the picture changes.

The instructor holds the fault-injection and fix commands in the answer key. Diagnose it yourself first.

Brief it back

Explain each of these aloud before moving on. If you cannot, re-read the primer and re-run the verification commands.

- Brief the whole architecture layer by layer: how the IS-IS underlay feeds the SPF result, how that result drives the colored transport planes and the SID label tables, and how the L3VPN service resolves onto that transport. Make the dependency of each layer on the one below it explicit.
- Explain why diagnosing in strict layer order is the only reliable approach to a multi-fault scenario, using this capstone as the example: which lower fault masked which higher one, and why the service-layer evidence could not be trusted until the transport under it was proven clean.
- Describe how you would communicate this root cause to a customer: what you would say the impact was, what each fault was in plain language, how you confirmed the fix, and what reassurance you would give about detection and prevention going forward.